UNIDAD 6
VISTAS

FICHA TÉCNICA DE LA UNIDAD DIDÁCTICA

Unidad Didáctica	Unidad Didáctica 9: Vistas en MySQL (Diseño Físico y Seguridad Lógica)
Resultados de Aprendizaje	**RA3:** Realiza el diseño físico de bases de datos utilizando asistentes, herramientas gráficas y el lenguaje de definición de datos.
Criterios de Evaluación	CE3.g, CE3.h, CE3.i, CE3.j
TiempoEstimado	6 Horas (Planificado para 1 semana de sesiones teórico-prácticas)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

1.1 ¿Qué es una vista?

2. OPERACIONES DDL SOBRE VISTAS

```
2.1 Creación
de Vistas (CREATE OR REPLACE VIEW)
2.1.1 Sintaxis
Completa y Parámetros del Motor
2.1.2 Algoritmos
de Procesamiento de Vistas
2.1.3 Definición de
Alias de Columnas
2.2 Eliminación de
Vistas (DROP VIEW)
2.3 Renombrado de
Vistas (RENAME TABLE)
2.4 Metadatos e
Inspección de Vistas
```

3. VISTAS ACTUALIZABLES

3.1 Restricciones de Modificación (INSERT, UPDATE, DELETE)

3.2 La Cláusula WITH CHECK OPTION

3.2.1 Prevención de Inconsistencias Lógicas

3.2.2 Diferencias entre CASCADED y LOCAL

4. CASOS DE USO AVANZADOS

4.1 Ejemplo 1: Seguridad Lógica en la Tabla Empleados

4.2 Ejemplo 2: Simplificación de Relaciones Complejas

4.3 Ejemplo 3: Prevención de Inconsistencias de Tipo

4.4 Ejemplo 4: Análisis de una Vista de Terceros No Actualizable

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

1.1 ¿Qué es una vista?

Una vista es una **tabla virtual** cuya definición lógica está almacenada de manera permanente en el diccionario de datos de la base de datos, pero que no posee datos propios físicamente almacenados en el disco. En su lugar, el motor de bases de datos recupera de forma dinámica el conjunto de resultados ejecutando la consulta SELECT que define la vista cada vez que esta se invoca en una sentencia DML.

En la arquitectura clásica de tres niveles ANSI-SPARC (Interno/Físico, Conceptual y Externo), las vistas constituyen la representación por excelencia del **nivel externo**.

Permiten aislar la percepción de los datos que tiene el usuario final o la aplicación de la estructura real lógica de las tablas base. Esto garantiza la **independencia lógica de los datos**: si el Administrador de Sistemas decide normalizar, dividir o reestructurar una tabla física subyacente, basta con redefinir la vista para que las aplicaciones de frontend sigan consumiendo la misma estructura sin requerir modificaciones en su código fuente.

Ventajas de las Vistas para el Administrador de Sistemas:

1. **Seguridad Lógica (Control de Accesos):** Permite restringir el acceso a datos altamente confidenciales. En lugar de otorgar privilegios globales sobre la tabla de empleados (que contiene campos como salario o comision), el SysAdmin puede crear una vista que excluya estas columnas y conceder privilegios exclusivamente sobre la vista.
2. **Simplificación de Consultas Complejas:** Centraliza joins complejos entre múltiples tablas, agregaciones matemáticas densas y filtros recurrentes en un solo objeto relacional. Los desarrolladores pueden interactuar con la vista como si fuera una tabla normal mediante consultas planas:

$$\text{SELECT} * \text{FROM vista_resumen WHERE campo} = \text{valor};$$

3. **Consistencia de Datos:** Asegura que todos los reportes de la infraestructura empresarial partan exactamente de la misma lógica unificada de cálculo.

2. OPERACIONES DDL SOBRE VISTAS

2.1 Creación de Vistas (CREATE OR REPLACE VIEW)

2.1.1 Sintaxis Completa y Parámetros del Motor

La creación de vistas en MySQL permite definir modificadores que influyen directamente en la optimización y en los privilegios de ejecución del motor.

```
CREATE
[OR REPLACE]
[ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
[DEFINER = user]
[SQL SECURITY { DEFINER | INVOKER }]
VIEW view_name [(column_list)]
AS select_statement
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

🔗 **Documentación Oficial: Consulta la sintaxis gramatical formal y todos los modificadores disponibles para este comando CREATE VIEW en [MySQL Reference Manual - CREATE VIEW Syntax]**

2.1.2 Algoritmos de Procesamiento de Vistas

El parámetro ALGORITHM determina el método físico que utilizará el Optimizador de MySQL para resolver la vista:

- **MERGE:** El motor combina el texto de la consulta del usuario con la definición de la vista en una única consulta unificada que se ejecuta sobre las tablas base. Es altamente eficiente porque permite al motor aprovechar de forma directa los índices de las tablas físicas subyacentes.
- **TEMPTABLE:** El motor ejecuta primero la consulta que define la vista, almacena temporalmente los resultados en una tabla intermedia en memoria (o disco) y luego ejecuta la consulta externa del usuario sobre dicha tabla intermedia. **Desventaja:** No puede utilizar índices de las tablas físicas y consume más memoria RAM.
- **UNDEFINED (Por defecto):** MySQL decide automáticamente qué algoritmo aplicar. Priorizará MERGE siempre que sea posible.

2.1.3 Definición de Alias de Columnas

Podemos renombrar de forma estructural las columnas resultantes directamente en la cabecera de la declaración, aislando por completo los nombres físicos del diseño relacional.

Ejemplo de uso práctico:

Si deseamos proyectar el personal de la oficina de Madrid ocultando sus atributos sensibles de salario, podemos definir alias en la cabecera del SELECT:

```
USE practica_dml;
CREATE OR REPLACE VIEW v_personal_madrid (ID, Empleado, Correo,
Ciudad) AS
SELECT
codigo_empleado,
CONCAT(nombre, ' ', apellido1),
email,
'Madrid'
FROM empleado
WHERE codigo_oficina = 'OFI-MAD';
```

2.2 Eliminación de Vistas (DROP VIEW)

Al eliminar una vista, únicamente se destruye su definición en el catálogo de metadatos. Las tablas físicas y los datos subyacentes permanecen inalterados.

```
DROP VIEW [IF EXISTS]
view_name [, view_name] ...
[RESTRICT | CASCADE];
```

Ejemplo de uso práctico:

```
DROP VIEW IF EXISTS v_personal_madrid;
```

2.3 Renombrado de Vistas (RENAME TABLE)

Dado que las vistas se comportan en gran medida como tablas lógicas en el espacio de nombres de la base de datos, se utiliza el comando estándar de renombrado de tablas para modificar su identificador en el esquema.

```
RENAME TABLE old_view_name TO new_view_name;
```

Ejemplo de uso práctico:

```
RENAME TABLE v_personal_madrid TO v_empleados_capital;
```

2.4 Metadatos e Inspección de Vistas

En un servidor corporativo con cientos de objetos, el administrador necesita auditar y verificar el catálogo físico de tablas para aislar exclusivamente las vistas del esquema y consultar su código fuente DDL de creación.

```
-- Listar todos los objetos indicando si son tablas base o
vistas
SHOW FULL TABLES;
-- Filtrar y mostrar únicamente las vistas del esquema activo
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

```
-- Consultar la sentencia exacta de creación (auditoría lógica)
SHOW CREATE VIEW v_empleados_capital;
```

3. VISTAS ACTUALIZABLES

3.1 Restricciones de Modificación (INSERT, UPDATE, DELETE)

MySQL permite realizar operaciones DML de modificación sobre una vista, impactando de forma directa y automática en las tablas base. Sin embargo, para que una vista sea **actualizable**, debe ser sencilla y directa.

La modificación **no está permitida** si la consulta de la vista contiene alguno de los siguientes elementos:

1. Funciones de agregación (SUM, MIN, MAX, AVG, COUNT).
2. Cláusulas de agrupación (GROUP BY o HAVING).
3. El modificador de unicidad DISTINCT.
4. Operadores de conjunto (UNION o UNION ALL).
5. Subconsultas en la cláusula de proyección SELECT.
6. Joins complejos que intenten actualizar más de una tabla de forma simultánea en una única sentencia DML.
7. Columnas calculadas mediante expresiones aritméticas o funciones escalares (no son actualizables sus valores derivados).

🔗 **Documentación Oficial: Consulta la documentación sobre las vistas actualizables en [MySQL Reference Manual - Updatable Views]**

3.2 La Cláusula WITH CHECK OPTION

3.2.1 Prevención de Inconsistencias Lógicas

Cuando realizamos una inserción o actualización mediante una vista que contiene una cláusula restrictiva (WHERE), es posible introducir un registro en la tabla física que no cumpla

con los propios criterios de filtrado de la vista, provocando que el nuevo dato "desaparezca" de la vista tras su creación.

Para evitar este comportamiento incoherente, se añade la cláusula WITH CHECK OPTION. El motor validará cualquier operación DML y rechazará con un error aquellas modificaciones que generen filas fuera del alcance del filtro de la vista.

3.2.2 Diferencias entre CASCADED y LOCAL

- **CASCADED (Por defecto):** Si la vista se ha construido sobre otras vistas, se evalúan de manera jerárquica y obligatoria los criterios y reglas lógicas de todas las vistas secundarias.
- **LOCAL:** Solo valida los criterios definidos de forma específica en la propia vista en cuestión, ignorando los filtros de las vistas base de las que se nutre si estas no cuentan de forma explícita con su propia restricción de chequeo.

4. CASOS DE USO AVANZADOS

A continuación se presentan una serie de ejemplos reales de administración utilizando de forma íntegra el esquema físico desarrollado en la Unidad 5 para resolver necesidades comunes de un SysAdmin.

4.1 Ejemplo 1: Seguridad Lógica en la Tabla Empleados

Requisito: Se necesita crear un objeto de acceso para el personal técnico que muestre información del empleado, su email, fecha de contratación y oficina, ocultando sus campos sensibles de nómina (salario y comision).

```
-- Creación de la vista segura
CREATE OR REPLACE VIEW v_empleados_publico AS
SELECT
codigo_empleado,
nombre,
apellido1,
email,
codigo_oficina,
fecha_contratacion
FROM empleado
WHERE activo = TRUE;
-- Consulta plana realizada por el usuario de soporte técnico
```

```
SELECT * FROM v_empleados_publico WHERE codigo_oficina =
'OFI-BAR';
```

Análisis técnico: Al conceder privilegios únicamente sobre v_empleados_publico y no sobre la tabla física empleado, se impide que usuarios con privilegios limitados conozcan el costo salarial corporativo.

4.2 Ejemplo 2: Simplificación de Relaciones Complejas

Requisito: El equipo de desarrollo de frontend necesita pintar de forma unificada el catálogo de productos con sus respectivos fabricantes asociados sin tener que escribir consultas INNER JOIN repetidamente en el código de la aplicación.

```
-- Creación de la vista integradora de inventario
CREATE OR REPLACE VIEW v_inventario_fabricantes AS
SELECT
p.codigo_producto AS Cod_Prod,
p.nombre AS Nombre_Producto,
p.precio_venta AS PVP,
p.gama AS Categoria,
f.nombre AS Fabricante,
f.pais AS Pais_Origen
FROM producto p
INNER JOIN fabricante f ON p.codigo_fabricante = f.codigo;
```

```
-- Consulta unificada directa desde la aplicación cliente
SELECT * FROM v_inventario_fabricantes WHERE PVP < 50.00;
```

Análisis técnico: Esta vista es un ejemplo de composición interna (INNER JOIN). **No es actualizable con INSERT** de forma directa ya que afecta a múltiples tablas (producto y fabricante) de manera simultánea en su proyección de origen.

4.3 Ejemplo 3: Prevención de Inconsistencias de Tipo

Requisito: El administrador debe delegar un acceso de modificación en cascada sobre los productos que pertenecen exclusivamente a la gama de 'Herramientas'. Debe garantizarse que, a través de dicho canal de edición, no sea posible agregar o renombrar un producto hacia otra gama distinta (como 'Ornamentales' o 'Frutales').

```
-- Creación de la vista restrictiva de gama
CREATE OR REPLACE VIEW v_gestion_herramientas AS
SELECT
codigo_producto,
nombre,
precio_venta,
gama,
codigo_fabricante
FROM producto
WHERE gama = 'Herramientas'
WITH CHECK OPTION;
-- Escenario A: Operación permitida (el registro cumple el filtro
gama = 'Herramientas')
INSERT INTO v_gestion_herramientas (codigo_producto, nombre,
precio_venta, gama, codigo_fabricante)
VALUES ('PROD-H99', 'Destornillador aislado de precisión', 12.50,
'Herramientas', 1);
-- Escenario B: Operación RECHAZADA (el registro viola la restricción
al ser 'Ornamentales')
INSERT INTO v_gestion_herramientas (codigo_producto, nombre,
precio_venta, gama, codigo_fabricante)
VALUES ('PROD-O99', 'Ficus Benjamina Gigante', 45.00, 'Ornamentales',
5);
-- Error esperado en el motor de base de datos:
-- ERROR 1369 (HY000): CHECK OPTION failed
'practica_dml.v_gestion_herramientas'
```

4.4 Ejemplo 4: Análisis de una Vista de Terceros No Actualizable

La base de datos Sakila de MySQL utiliza de forma extensa vistas para simplificar estructuras lógicas orientadas a la facturación y generación de informes de películas.

Analicemos la estructura sintáctica de la vista de producción customer_list:

```
CREATE VIEW customer_list AS
SELECT
cu.customer_id AS ID,
CONCAT(cu.first_name, ' ', cu.last_name) AS name,
a.address AS address,
a.postal_code AS `zip code`,
a.phone AS phone,
```

```
city.city AS city,
country.country AS country,
IF(cu.active, 'active', '') AS notes,
cu.store_id AS SID
FROM customer AS cu
JOIN address AS a ON cu.address_id = a.address_id
JOIN city ON a.city_id = city.city_id
JOIN country ON city.country_id = country.country_id;
```

Análisis técnico para debate y auditoría:

- **¿Es esta vista actualizable con un comando INSERT?** No, la vista utiliza funciones de manipulación de cadenas en campos lógicos calculados (como CONCAT para el campo name e IF para el campo notes), además de involucrar múltiples tablas relacionales mediante joins encadenados. Al violar las reglas de actualizabilidad, el motor físico impedirá cualquier acción DML de inserción a través de esta.